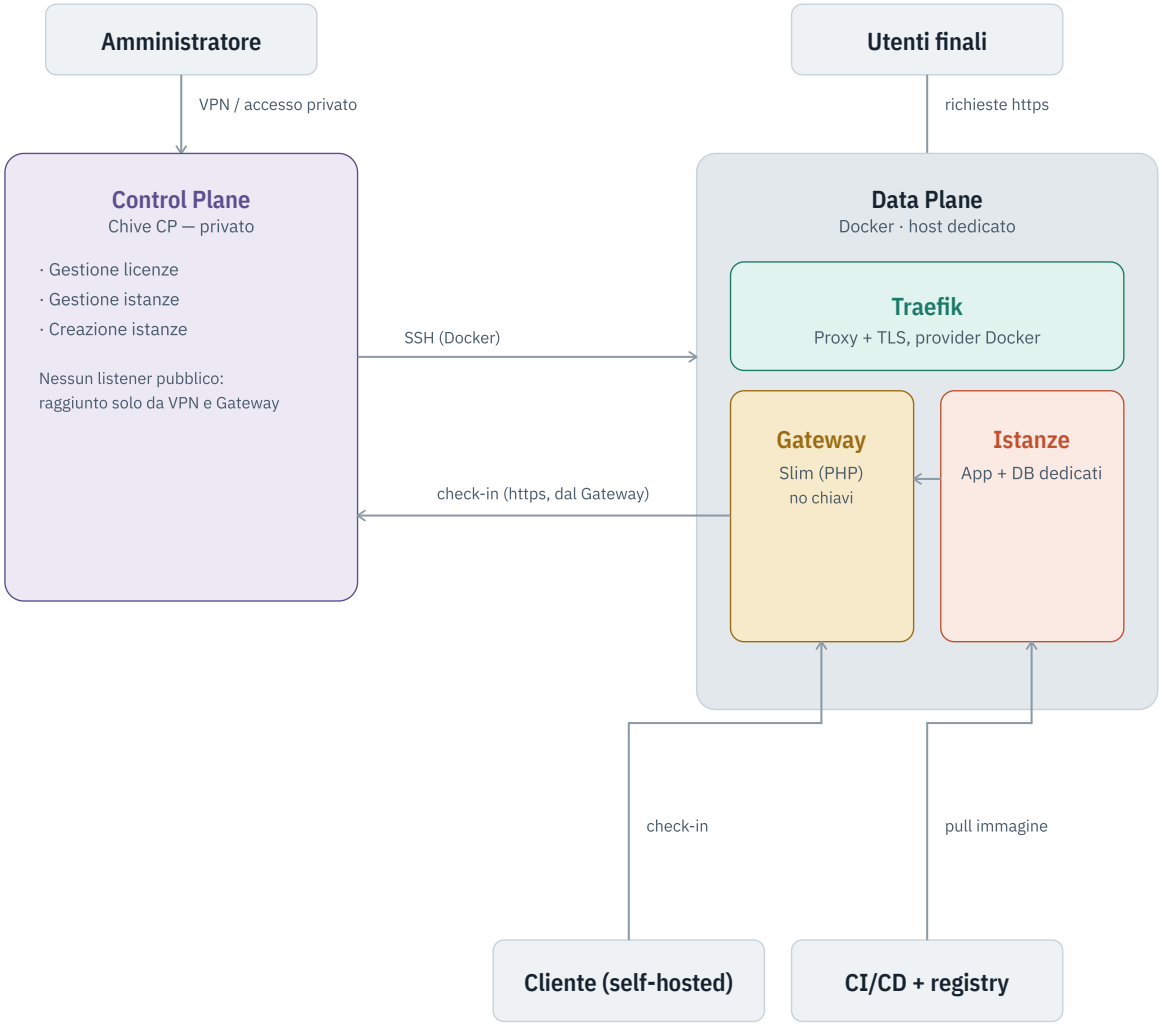
Provisioning istanze Docker

Chi fa cosa quando nasce una nuova istanza: dall'amministratore che preme il pulsante fino al container che serve il traffico.

INDICE

1. Architettura (control plane / gateway / data plane)
2. Responsabilità per attore
3. Funzionalità del control plane
4. Funzionalità del gateway
5. Funzionalità del data plane
6. Chi avvia la creazione di un'istanza — amministratore vs richiesta demo
7. Come funzionano i comandi verso il data plane
8. Context Docker e Data Plane multipli
9. Tabella dei secret
10. Come nasce la licenza — managed vs self-hosted
11. Il pacchetto per il cliente self-hosted
12. Pipeline di build e rilascio
13. Aggiornamento istanza e ripristino da backup
14. Struttura cartelle: data plane e control plane

SCHEMA · ARCHITETTURA (CONTROL PLANE / GATEWAY / DATA PLANE) rev. 2026-09-17



Control plane (privato) · Gateway (dentro il Data Plane, pubblico minimale) · Proxy / TLS

Istanze (workload) · Infrastruttura / attori esterni

Il Gateway non è una macchina a sé: è un container in più sul Data Plane, instradato da Traefik esattamente come le istanze. Il Data Plane era già l'unica macchina esposta pubblicamente e senza chiavi permanenti, il profilo giusto per ospitarlo.

Responsabilità per attore

Control Plane

CHIVE CP · PRIVATO, NESSUN LISTENER PUBBLICO

- Gestione licenze
- Gestione istanze esistenti
- Creazione di nuove istanze

Gateway

PUBBLICO · MINIMALE, NESSUNA CHIAVE

- Riceve i check-in delle istanze (managed e self-hosted)
- Riceve il webhook della pipeline CI
- Riceverà anche le richieste demo pubbliche (futuro)
- Inoltra al Control Plane su canale privato

Traefik

CONTAINER DOCKER, SUL DATA PLANE

- Instrada il traffico per dominio via label
- Emette TLS automaticamente

Istanze

DEV, STAGING, CLIENTE/DEMO

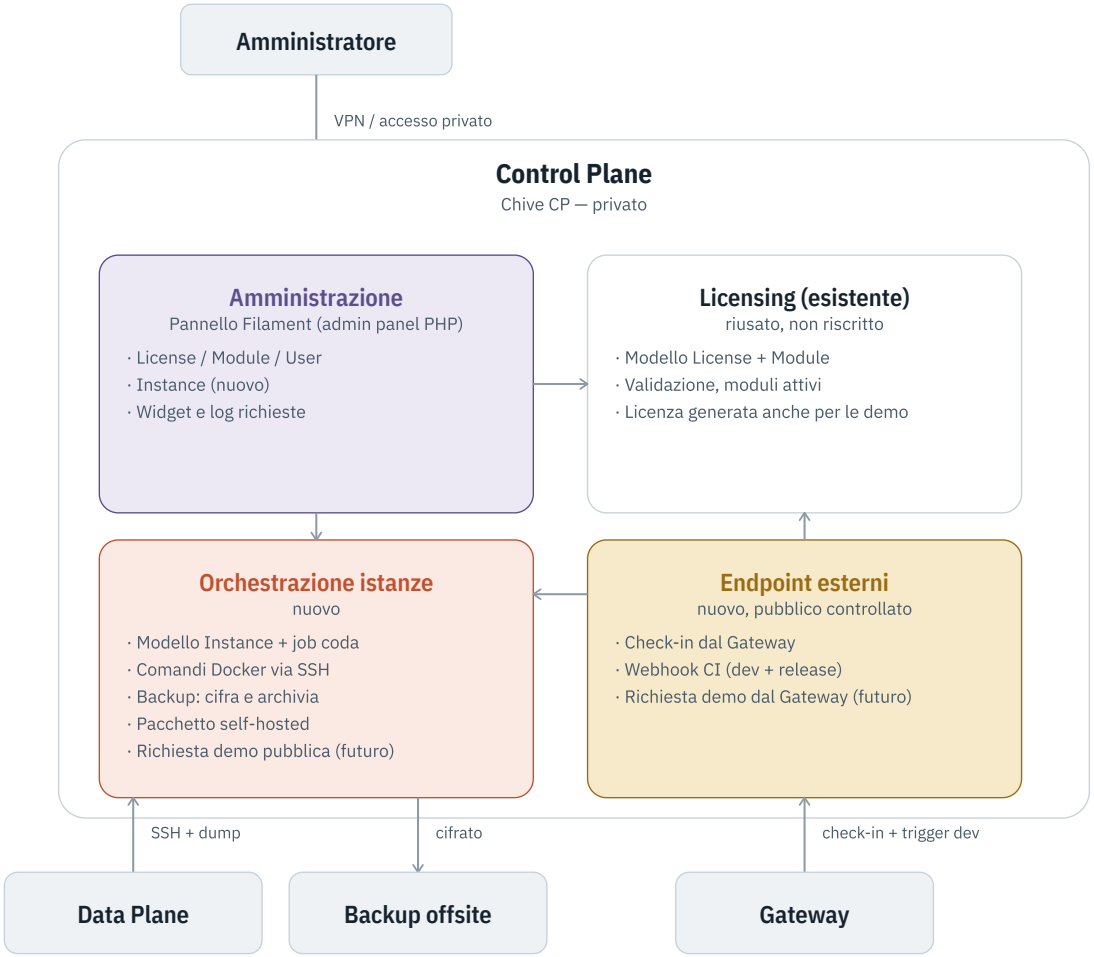
- App + DB dedicati per istanza
- Stesso meccanismo per tutte

CI/CD + registry

PIPELINE ESTERNA

- Build dell'immagine versionata
- Pubblicazione sul registry

SCHEMA · FUNZIONALITÀ DEL CONTROL PLANE rev. 2026-09-17

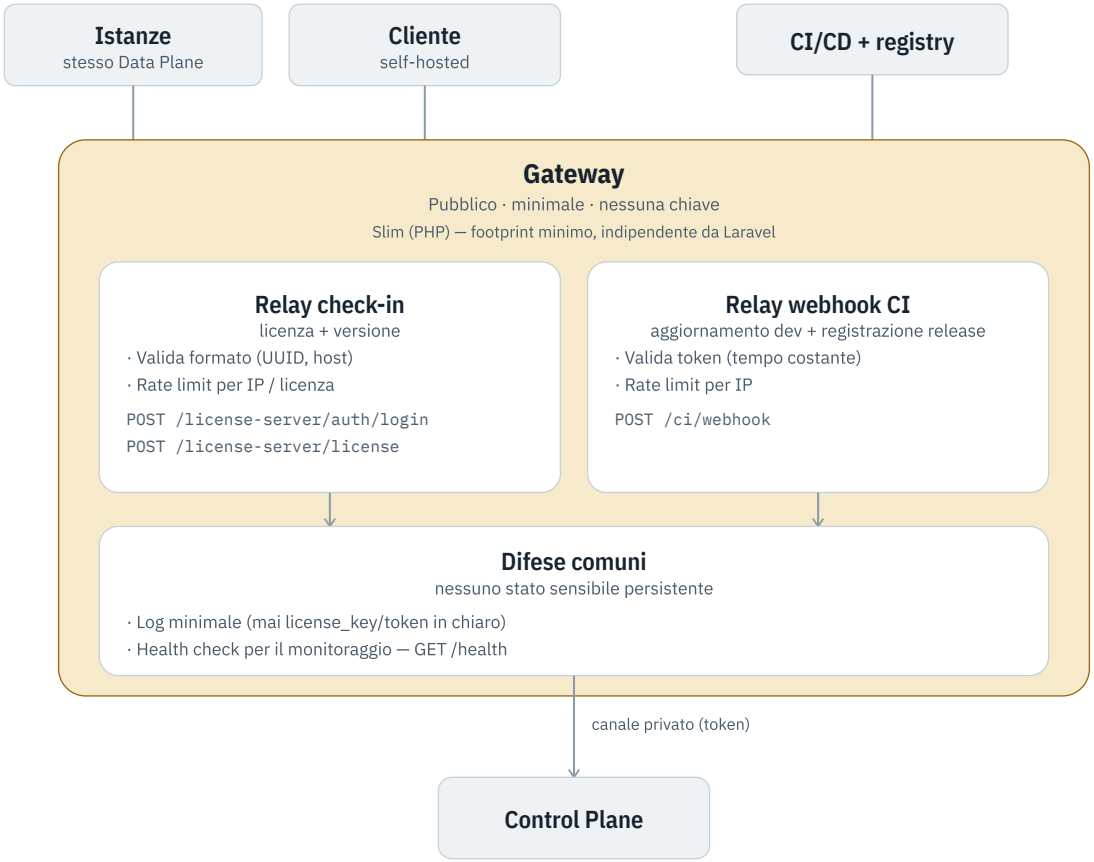


Amministrazione (Filament) Licensing (già esistente) Orchestrazione istanze (nuovo)

Endpoint esterni (nuovo)

Il dump arriva dal Data Plane via SSH e viene cifrato **subito** lato Control Plane, prima di toccare qualunque disco — non resta mai, nemmeno temporaneamente, sul Data Plane. La copia offsite riceve solo il file già cifrato. **Nessun listener pubblico**: anche il webhook CI passa dal Gateway, non arriva diretto. "Data Plane" e "Gateway" qui sotto sono due canali distinti (SSH per Docker/dump, HTTPS per il check-in) verso la **stessa macchina fisica** — il Gateway gira lì come container, non su un server a parte.

SCHEMA · FUNZIONALITÀ DEL GATEWAY rev. 2026-09-17



Gateway (contenitore, pubblico) Funzioni interne / Control Plane

Il Gateway gira come **container sul Data Plane**, instradato da Traefik — non una terza macchina: il Data Plane era già l'unica esposta pubblicamente e senza chiavi permanenti. Il "canale privato" verso il Control Plane in fondo a questo schema è quindi l'unico salto che attraversa davvero una rete tra macchine diverse. Scritto in **Slim** (PHP) invece di Laravel: stesso linguaggio del resto dello stack, ma un nucleo molto più piccolo — solo routing e gestione richieste, niente ORM/code/sessioni installati per forza. Per un componente con due sole rotte reali, meno dipendenze da tenere aggiornate e nessuna vulnerabilità di Laravel/Chive CP che si ripercuote automaticamente qui. Se il Control Plane non risponde in tempo, il Gateway restituisce un errore chiaro al chiamante invece di restare appeso. **Cosa significa "nessuna cache di riserva"**: il Gateway potrebbe tenere una copia recente degli esiti di validazione, per rispondere comunque a un check-in anche se il Control Plane fosse irraggiungibile per un momento — abbiamo scelto di *non* farlo nella v1: un check-in fallito è solo un errore temporaneo (il prossimo, un'ora dopo, riprova), mentre una cache sarebbe un'altra copia di dati da tenere sincronizzata e protetta anche sul Gateway — esattamente il tipo di stato in più che vogliamo evitargli.

Un terzo relay è già previsto ma non ancora costruito: quando esisterà la richiesta demo pubblica (vedi "Chi avvia la creazione di un'istanza"), sarà il Gateway a riceverla dalla landing page e a inoltrarla al Control Plane — stessa logica dei due relay qui sopra, nessun meccanismo nuovo da progettare.

Tutti gli endpoint del Gateway, con i body reali che viaggiano in richiesta e risposta.

POST /license-server/auth/login — chi chiama: l'istanza stessa, al check-in.

```
// richiesta
{
  "domain": "acme-corp.thecustomerhive.com",
  "license_key": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}

// risposta (200)
{
  "token": "1|xY9kLp3mZqRt7vWn2sHj8cBf...",
  "token_type": "Bearer"
}

// risposta (401 — dominio/chiave sbagliati o IP non corrispondente)
{ "message": "Unauthenticated." }
```

POST /license-server/license — chi chiama: l'istanza, subito dopo aver ottenuto il token. Nessun corpo in ingresso, solo l'header `Authorization: Bearer <token>`.

```
// risposta (200)
{
  "license_key": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "domain": "acme-corp.thecustomerhive.com",
  "status": "active",
  "expires_at": "2027-09-24",
  "modules": ["crm", "invoicing", "reports"]
}
```

POST /ci/webhook — chi chiama: la pipeline CI, non un'istanza.

```
// richiesta (ramo dev)
{ "tag": "dev-a1b2c3d", "branch": "dev" }

// richiesta (ramo release)
{ "tag": "v1.4.0", "branch": "release" }

// risposta (202 — l'azione vera avviene dentro Chive CP, non nella risposta)
{ "received": true }
```

GET /health — chi chiama: un sistema di monitoraggio esterno. Nessun corpo in ingresso, nessuna autenticazione — non inoltra nulla a Chive CP, verifica solo che il processo del Gateway sia vivo.

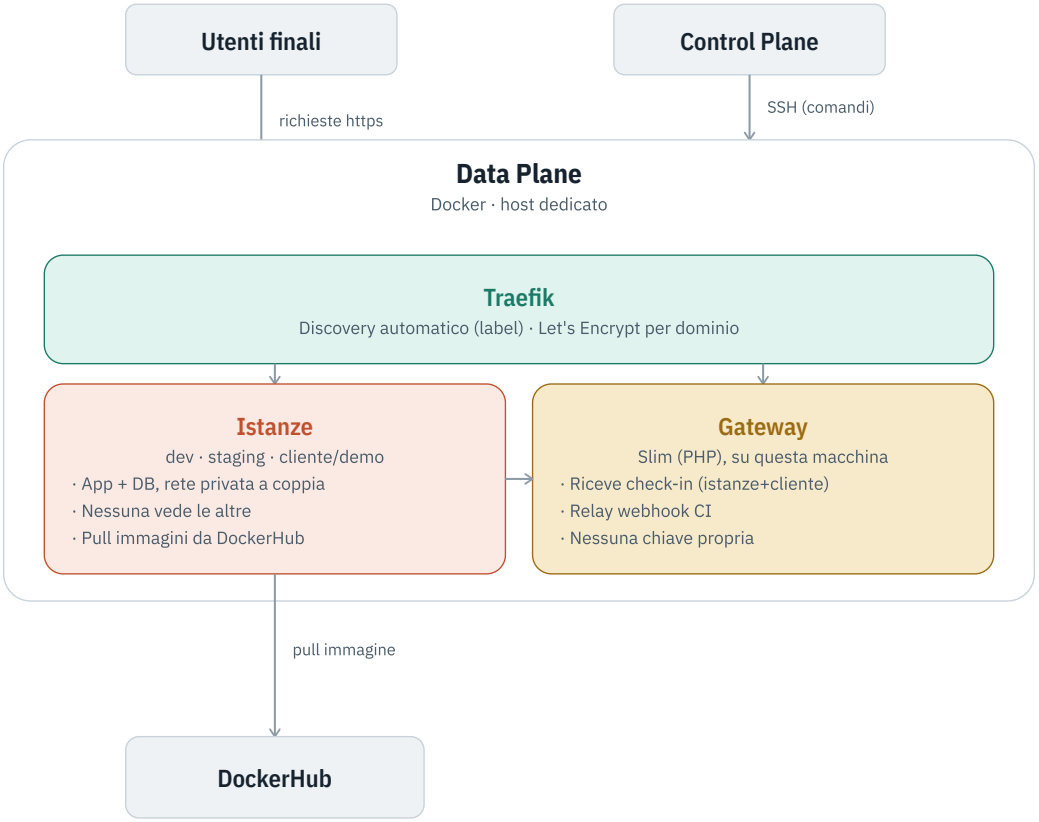
```
// risposta (200)
{ "status": "ok" }
```

POST /demo-request — futuro, non ancora costruito. Chi chiamerà: il modulo "Preparami una demo" sulla landing page.

```
// richiesta
{
  "company": "Acme Corp",
  "email": "mario.rossi@acmecorp.it"
}

// risposta (200)
{ "message": "Richiesta ricevuta, ti contatteremo a breve." }
```

SCHEMA · FUNZIONALITÀ DEL DATA PLANE rev. 2026-09-17



Traefik Istanze Gateway Attori esterni

Ogni istanza sta sulla propria rete Docker privata (app↔DB); solo il container app si aggancia anche alla rete condivisa con Traefik — il DB non è mai raggiungibile da un'altra istanza, nemmeno sulla rete interna. Il Gateway è un container in più sulla stessa macchina, instradato dallo stesso Traefik: il check-in da un'istanza verso il Gateway è una chiamata interna al Data Plane, non un salto di rete verso un'altra macchina. Nessuna porta oltre 80/443/22 è esposta sull'host.

Amministratore

OGGI — PRIORITÀ ATTUALE

1. Compila il modulo "Instance" nel pannello Filament (nome cliente, Data Plane su cui farla nascere, ecc.) — Chive CP propone già uno slug valido, modificabile
2. Il record Instance nasce già "approvato" — è lui stesso l'amministratore a deciderlo
3. Parte subito il job di creazione in coda — gli stessi comandi della sezione successiva
4. Da qui in poi: stesso ciclo di vita di ogni istanza (check-in, aggiornamenti, backup)

Richiesta demo pubblica

FUTURO — NON ANCORA COSTRUITO

1. Un visitatore compila il modulo "Preparami una demo" sulla landing page (nome azienda, non uno slug)
2. La richiesta passa dal **Gateway** (Chive CP non ha listener pubblico) e crea un record Instance con stato "in attesa", con uno slug generato per intero da Chive CP
3. L'amministratore la vede nel pannello e la approva con un click
4. Da quel click in poi: **lo stesso identico job** di creazione della modalità amministratore — nessun meccanismo nuovo
5. Da qui in poi: stesso ciclo di vita di ogni istanza (check-in, aggiornamenti, backup)

Le due modalità condividono lo stesso motore: cambia solo **chi/cosa crea il record Instance e come viene validato prima di entrare in coda**, non come l'istanza viene poi effettivamente creata. Per questo la richiesta demo pubblica si potrà aggiungere in un secondo momento senza toccare nulla di quanto già costruito per l'amministratore — è solo un secondo innesco verso lo stesso job. Il click di approvazione non è burocrazia fine a sé stessa: finché esiste, un modulo pubblico raggiungibile da chiunque non ha bisogno di anti-abuso sofisticato (rate limit aggressivi, verifica email, CAPTCHA) per restare sicuro, perché nessun container nasce davvero senza che un amministratore l'abbia guardato — un'approvazione automatica, se mai servirà, potrà arrivare più avanti come passo successivo, non come condizione per partire.

Lo slug (es. `acme-corp`) non è mai testo libero: finisce nel sottodominio pubblico, nel nome del progetto Docker (`-p <slug>` in ogni comando della sezione successiva) e nei percorsi di due macchine diverse — un carattere sbagliato lì dentro può rompere silenziosamente uno di questi

usi. Per questo è **Chive CP a generarlo**, non chi scrive il modulo: lo calcola dal nome digitato (solo minuscolo, lettere/numeri/trattini), controlla che non sia già in uso e aggiunge da sola un suffisso (`-2` , `-3` ...) se serve. Per l'amministratore è un suggerimento modificabile, ma sempre dentro queste regole; per la richiesta demo pubblica il visitatore non sceglie nulla — lo slug è generato per intero da Chive CP, e resta comunque correggibile dall'amministratore al momento dell'approvazione.

Tutte le operazioni sotto avvengono solo tramite un job in coda, mai come azione istantanea al click. Se un passaggio fallisce, si ferma lì — nessun tentativo automatico di ripeterlo. `dataplane` negli esempi è il nome di un context — vedi la sezione successiva per cosa significa.

Creazione istanza — fa partire i container, poi prepara le tabelle del database.

```
docker --context dataplane compose -f /opt/chivecp/instances/<slug>/docker-compos
docker --context dataplane compose -p <slug> exec -T app php artisan migrate --fo
```

Verifica che sia pronta — ripete questo controllo ogni 5 secondi finché non risulta `healthy`, o finché non scade un tempo massimo (es. 60 secondi): non è un nuovo tentativo dopo un fallimento, è solo l'attesa che il normale avvio del container finisca.

```
docker --context dataplane inspect --format='{{.State.Health.Status}}' <slug>-app
```

Se la creazione fallisce (la migration fallisce, o l'healthcheck non diventa mai `healthy` entro il tempo massimo) — nessun rollback da fare, perché non esiste ancora nessun dato reale da proteggere: il job si limita a **fermare** i container (stessa azione di "Stop" qui sotto) e segna l'istanza come "creazione fallita", senza cancellare nulla in automatico. Restano lì, ferme, finché l'amministratore non ispeziona i log per capire cosa non ha funzionato — solo a quel punto decide se eliminarla per sempre e riprovare da zero, o lasciarla così com'è.

Stop (reversibile) — ferma i container, i dati salvati restano intatti.

```
docker --context dataplane compose -f /opt/chivecp/instances/<slug>/docker-compos
```

Eliminazione definitiva — stessa cosa, ma rimuove anche i dati salvati.

```
docker --context dataplane compose -f /opt/chivecp/instances/<slug>/docker-compos
```

Aggiornamento — scarica la nuova versione e riavvia solo il container dell'app, il database resta acceso.

```
docker --context dataplane compose -f /opt/chivecp/instances/<slug>/docker-compos
docker --context dataplane compose -f /opt/chivecp/instances/<slug>/docker-compos
docker --context dataplane compose -p <slug> exec -T app php artisan migrate --fo
```

Backup — estrae, comprime e cifra in un unico flusso continuo con una chiave dedicata a questa istanza, mai in chiaro su disco.

```
docker --context dataplane compose -p <slug> exec -T db sh -c 'mysqldump -u root
  | gzip \
  | openssl enc -aes-256-cbc -salt -pbkdf2 -k "$BACKUP_ENCRYPTION_KEY" \
  -out /tmp/<slug>_$(date +%F).sql.gz.enc
```

Ripristino — la stessa catena al contrario, per tornare a un backup precedente.

```
openssl enc -d -aes-256-cbc -pbkdf2 -k "$BACKUP_ENCRYPTION_KEY" -in <slug>_backup
  | gunzip \
  | docker --context dataplane compose -p <slug> exec -T db sh -c 'mysql -u root
```

Cosa significa `--context dataplane` nei comandi appena visti, e come Chive CP gestisce più di un Data Plane.

Chive CP può gestire **più Data Plane** — alla creazione di un'istanza, l'amministratore sceglie da un elenco su quale farla nascere, e quella scelta resta scritta nel fascicolo dell'istanza per tutti i comandi futuri.
Cos'è di preciso un "context": è solo un nome comodo, salvato una volta su Chive CP, che tiene la corrispondenza "questo nome → questa macchina, raggiunta così". Non è il Data Plane in sé — è l'indicazione per trovarlo. Ogni Data Plane registrato ha il suo, distinto dagli altri: quando un comando dice `--context dataplane-2`, Chive CP va a guardare quella corrispondenza salvata e sa subito a quale macchina indirizzare l'ordine, senza doverlo cercare o indovinare ogni volta.
Registrazione di un nuovo Data Plane — fatta una volta sola, quando se ne aggiunge uno (non ad ogni istanza).

```
docker context create dataplane-2 --docker "host=ssh://deploy@10.0.4.12"
```

Elenco dei Data Plane registrati — è la lista da cui l'amministratore sceglie alla creazione di un'istanza.

```
docker context ls
```

Il file `docker-compose.yml` (e il `.env` a fianco) **non lasciano mai Chive CP** — nessun trasferimento verso il Data Plane. Chive CP li legge in locale e li trasforma in istruzioni già complete ("crea un container con queste caratteristiche, con queste variabili già dentro") che manda al Docker del Data Plane attraverso il context: è quello a eseguirle davvero, senza aver mai bisogno di una sua copia dei file. I due file restano sul disco di Chive CP per tutta la vita dell'istanza — servono di nuovo a ogni aggiornamento futuro — e vengono cancellati solo quando elimini l'istanza per sempre. L'unica accortezza: le cartelle da montare (upload, dati del database) vanno scritte nel file come percorsi validi **sul Data Plane**.

Elenco progressivo di ogni secret coinvolto nel sistema, aggiornato via via che si approfondiscono le varie parti.

SECRET	DOVE VIVE	DOVE/COME VIENE CREATO	A COSA SERVE
MYSQL_ROOT_PASSWORD	nel `.env` dell'istanza, letto dal container del database	generata casualmente da Chive CP alla creazione dell'istanza — letta dall'immagine MySQL al primo avvio, che la usa da sola per impostare la password di root	autentica come root dentro il database — usato in Backup per estrarre il dump e in Ripristino per reimportarlo
DB_USERNAME / DB_PASSWORD	nel `.env` dell'istanza, letto dal container del database e da quello dell'app	generata casualmente da Chive CP alla creazione dell'istanza, insieme a MYSQL_ROOT_PASSWORD — letta dall'immagine MySQL al primo avvio, che crea da sola l'utente dedicato con accesso solo al database di quell'istanza	l'app si collega al proprio database con queste, mai con la password di root — meno privilegi, meno danno se l'app viene compromessa
BACKUP_ENCRYPTION_KEY	colonna cifrata sul record `Instance`, nel database di Chive CP — non un file: non serve a nessun container, quindi non è mai nel `.env`, e non esiste sul Data Plane	generata casualmente da Chive CP alla creazione dell'istanza, come le due sopra — **una chiave diversa per ogni istanza**, non condivisa; cifrata nel database con la stessa `APP_KEY` applicativa	cifra il dump in Backup e lo decifra in Ripristino — senza questa chiave un backup rubato è inutilizzabile, e compromette solo quell'istanza, non le altre
Chiave privata SSH di Chive CP	solo su Chive CP, come utente di sistema — la chiave pubblica corrispondente sta negli `authorized_keys` dell'utente `deploy` su ogni Data Plane	generata una tantum quando si prepara Chive CP — condivisa da tutti i comandi verso tutti i Data Plane, non per singola istanza	autentica ogni connessione SSH aperta da un context — è quello che rende possibile ogni comando `docker --context ...` visto finora

SECRET	DOVE VIVE	DOVE/COME VIENE CREATO	A COSA SERVE
$CI_WEBHOOK_TOKEN	nei "Secrets" di GitHub Actions (mai nel codice della pipeline) e sul Gateway/Chive CP, per validarlo in arrivo	generato una tantum quando si configura la pipeline — condiviso da entrambi i rami (push su dev e tag di release), non per singola istanza	autentica la chiamata `POST /ci/webhook` della pipeline verso il Gateway — senza un token valido la richiesta viene scartata

Managed

ISTANZA SU INFRASTRUTTURA NOSTRA

1. Chive CP genera la licenza per il dominio
2. La chiave finisce nel file `.env` dell'istanza
3. Chive CP stessa fa partire il container (SSH sul Data Plane)
4. Al primo avvio l'app trova la chiave e se la registra da sola
5. Da qui in poi: check-in orario, come ogni installazione

Self-hosted

ISTANZA SU INFRASTRUTTURA DEL CLIENTE

1. Chive CP genera la licenza per il dominio (stessa funzione)
2. La chiave finisce nel file `.env` del pacchetto
3. Il pacchetto va all'IT del cliente, che lo scarica
4. Il cliente fa partire il container sulla propria macchina
5. Al primo avvio l'app trova la chiave e se la registra da sola (stesso meccanismo)
6. Da qui in poi: check-in orario verso il Gateway — l'unico segnale che abbiamo

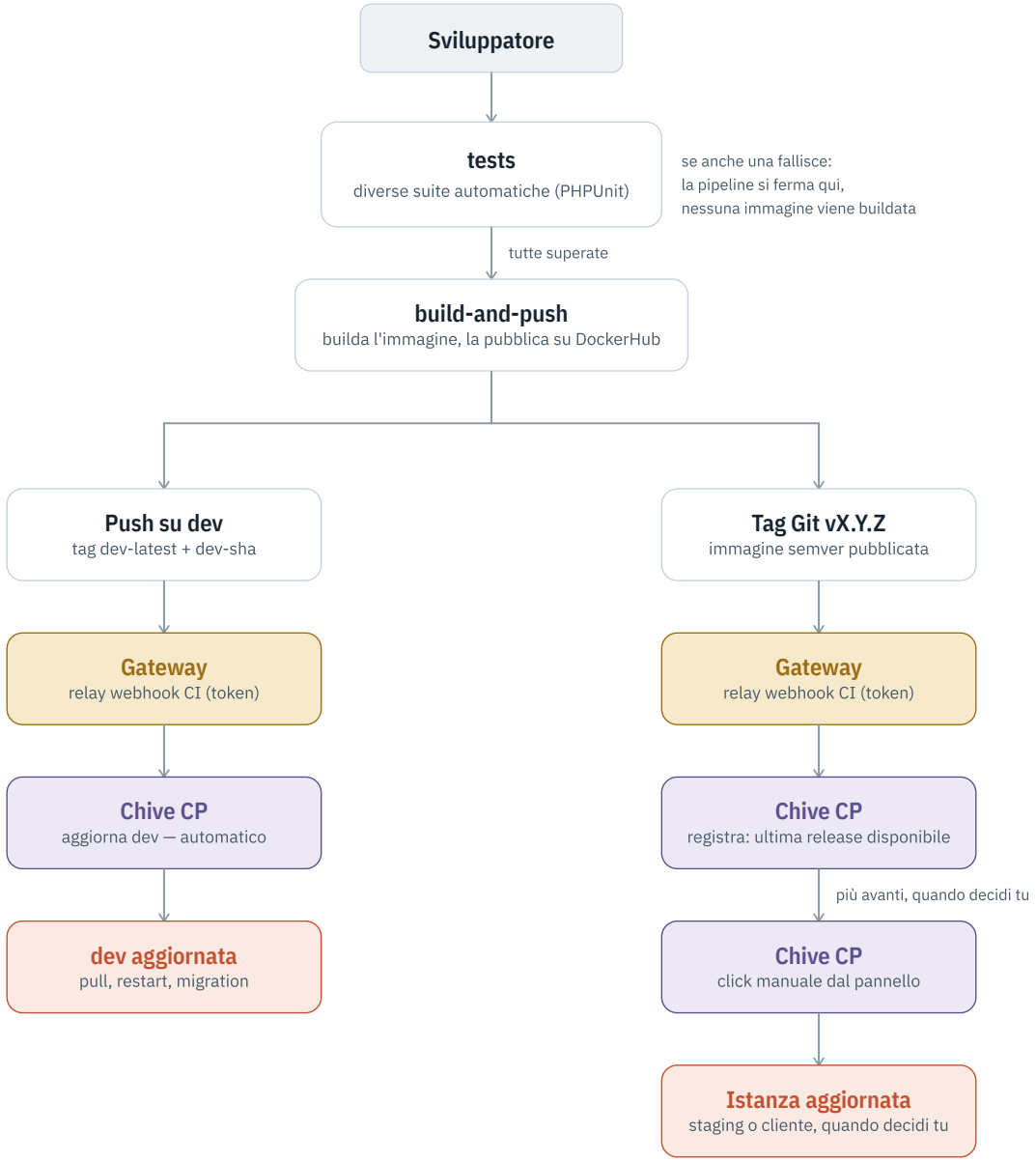
Il meccanismo di attivazione è **identico** nei due casi: la chiave arriva sempre via file di configurazione, e l'app se la registra da sola al primo avvio — nessuna schermata da compilare a mano. L'unica vera differenza è **chi fa partire il container**: noi (managed) o il cliente (self-hosted). Il "check-in orario" descritto sopra non è l'unico momento di validazione: è un comportamento già esistente oggi che la licenza venga controllata anche in altri punti, ad esempio quando si crea un nuovo tenant o utente dentro l'istanza — questo schema si concentra sul check-in periodico perché è quello rilevante per sapere se un'istanza è ancora viva e su che versione gira, non perché sia l'unico controllo che esiste.

SCHEMA · IL PACCHETTO PER IL CLIENTE SELF-HOSTED rev. 2026-09-17

```
pacchetto-<slug>/
├── docker-compose.yml        app + db, immagine pinnata alla versione
├── .env                      LICENSE_KEY, APP_KEY, credenziali DB — gi
└── ISTRUZIONI.txt            comando da lanciare, requisiti minimi
```

Solo tre file, niente di più: nessuna cartella `storage/` o `db-data/` nel pacchetto — Docker le crea vuote da sé al primo avvio, sulla macchina del cliente. Il file `.env` è già compilato da Chive CP (licenza, chiave applicativa, credenziali del database generate per quell'installazione) — il cliente non deve scrivere né decidere nulla, solo eseguire il comando indicato in `ISTRUZIONI.txt`.

SCHEMA · PIPELINE DI BUILD E RILASCIO rev. 2026-09-17



Gateway Chive CP Istanza aggiornata Fasi della pipeline

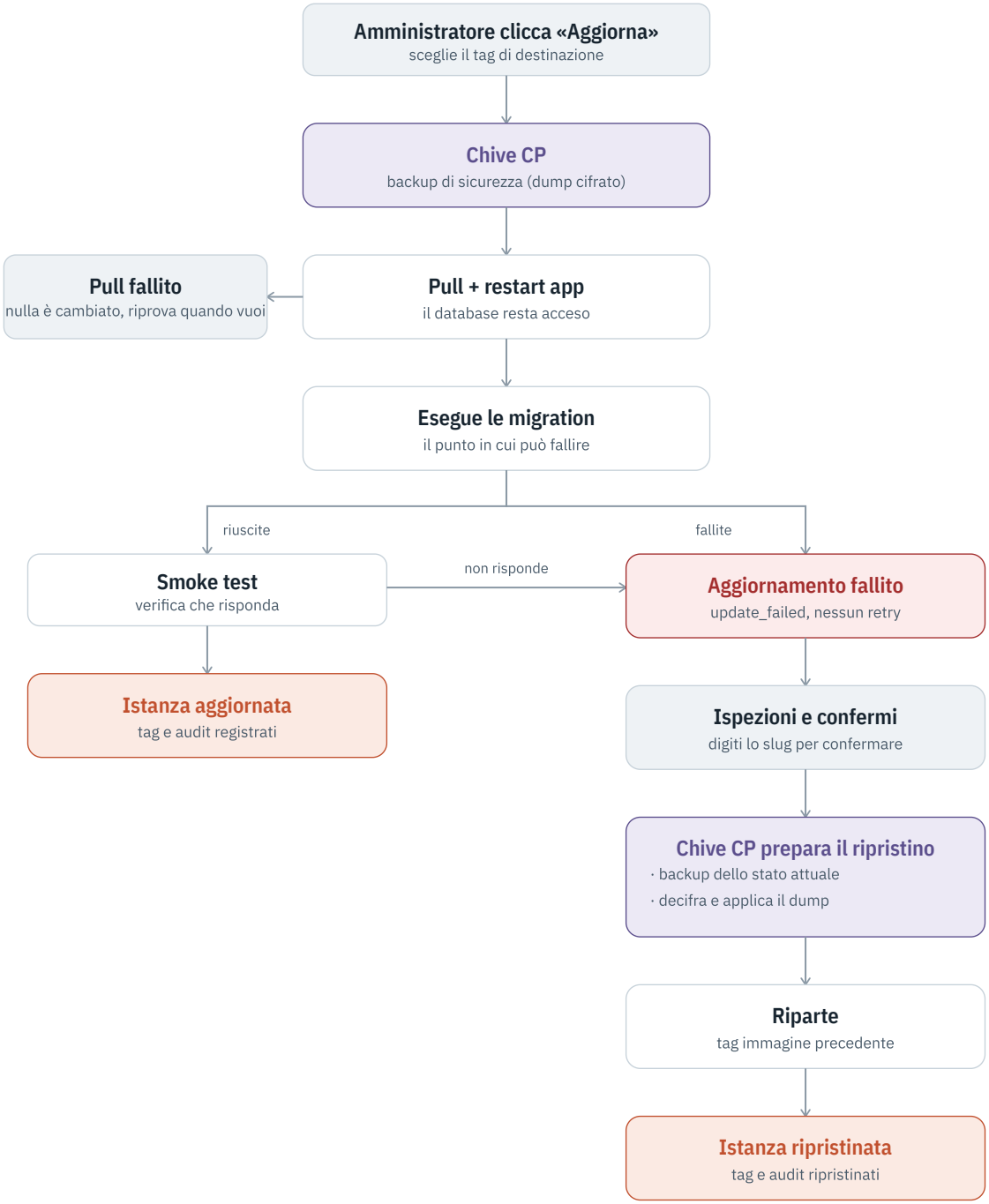
Il ramo sinistro (dev) parte da solo a ogni push. Il ramo destro (release) parte solo quando **tu** crei un tag Git — nessuna immagine semver arriva mai a staging o a un cliente senza un'azione esplicita. In entrambi i casi la pipeline (GitHub Actions, che gira fuori dalla nostra infrastruttura) non può chiamare Chive CP direttamente — non ha alcun listener pubblico — quindi passa sempre dal **Gateway**, con lo stesso token di servizio usato per il webhook di dev. Questo valore è quello con cui Chive CP confronta la versione in uso su ogni istanza per segnalare chi è rimasto indietro: per le managed lo sa già da sé — è stata lei a eseguire l'ultimo aggiornamento, quindi il proprio

fascicolo dice già che tag gira; per le self-hosted invece il check-in è l'unica fonte, perché è il cliente a decidere se e quando applicare l'aggiornamento, e Chive CP non ha altra visibilità su cosa sia realmente in esecuzione.

Come la pipeline chiama davvero il webhook — non c'è nulla di speciale: GitHub Actions esegue la pipeline su una macchina temporanea con accesso a internet come qualunque altra, quindi l'ultimo passo del job (subito dopo il push riuscito dell'immagine) è solo una richiesta HTTP in uscita, la stessa cosa che farebbe un browser.

```
curl -X POST https://gateway.thecustomerhive.com/ci/webhook \
  -H "Authorization: Bearer $CI_WEBHOOK_TOKEN" \
  -d '{"tag":"dev-a1b2c3d"}'
```

Il Gateway riceve questa richiesta come una qualsiasi altra sulla sua rotta pubblica `POST /ci/webhook` — non sa né gli interessa che ad averla mandata sia stato uno script invece di una persona. Il token (`$CI_WEBHOOK_TOKEN`) non è mai scritto nel codice della pipeline: vive una volta sola nei "Secrets" di GitHub Actions, e viene inserito automaticamente come variabile d'ambiente solo durante l'esecuzione — così non compare mai in chiaro nel repository.



Chive CP · Fallimento · Istanza (esito) · Fasi del processo / azioni manuali

Il pull che fallisce è il caso più innocuo: succede prima che il container dell'app venga toccato, quindi quello vecchio sta ancora girando, intatto — non serve nessun ripristino, basta segnare il tentativo come fallito e riprovare quando vuoi. **Migration e smoke test falliti** finiscono invece nello stesso percorso di ripristino, anche se solo la migration cambia davvero lo schema del

database: tenerli uniti evita di dover costruire e verificare due percorsi diversi, e riporta sempre a una combinazione immagine+database di cui sai già che funzionano insieme. Il ripristino non usa "l'immagine precedente" da sola: applica di nuovo il **database** del backup pre-aggiornamento, perché MySQL non annulla da solo una modifica di struttura a metà — poi riporta anche il tag immagine coerente con quei dati. Questo ramo non parte mai da solo: dopo un aggiornamento fallito tutto si ferma e aspetta — solo l'avvio del ripristino richiede una tua azione esplicita (ispezione + conferma), poi Chive CP esegue il resto in automatico, come un job qualunque. La conferma richiede di **digitare lo slug** dell'istanza, non un semplice "Sì": un click per abitudine è facile su un "Sei sicuro?", mentre scrivere il nome esatto ti costringe a verificare consapevolmente su quale istanza stai per sovrascrivere il database — l'ultima rete contro un click sulla riga sbagliata, prima che diventi irreversibile.

SCHEMA · STRUTTURA CARTELLE: DATA PLANE E CONTROL PLANE rev. 2026-09-17

```
/opt/customerhive/                          sul Data Plane
├── traefik/                               proxy Docker — persistente tra i redeploy
│   ├── docker-compose.yml
│   └── letsencrypt/                       certificati TLS
└── instances/
    ├── dev/                               stesso meccanismo di un'istanza cliente
    │   ├── storage/                       upload dell'app
    │   └── db-data/                       dati MySQL
    ├── staging/                           stessa struttura di dev/
    └── <slug-cliente>/                    istanza cliente/demo — stessa struttura
```

Solo dati, nessun file di configurazione: `docker-compose.yml` e `.env` non stanno qui — restano su Chive CP (vedi sotto) e non vengono mai trasferiti, come già spiegato. Nessun backup qui: se il Data Plane viene compromesso o perso, non c'è nulla di cifrato da recuperare su questa macchina — solo container e dati correnti.

```
/opt/chivecp/                               sul Control Plane
└── instances/
    ├── dev/
    │   ├── docker-compose.yml                  letto da Chive CP, mai inviato al Da
    │   └── .env                                licenza, credenziali, chiave app
    ├── staging/
    └── <slug-cliente>/
```

Ogni cartella qui rispecchia una in `instances/` sul Data Plane, stesso slug — ma il contenuto è complementare: qui la configurazione, là i dati. Restano entrambe finché l'istanza esiste, e vengono rimosse insieme solo con l'eliminazione definitiva.

```
storage offsite/                                  non su Chive CP, non sul Data Plane
└── backups/
    └── <slug-cliente>/
        └── <slug-cliente>_2026-09-17.sql.gz.enc    dump già cifrato prima di lasc
```

Chive CP non trattiene una copia locale del backup: cifra il dump appena arriva e lo inoltra subito allo storage offsite, senza conservarne un'altra copia sul proprio disco — meno dati sensibili fermi su una singola macchina. **Chiusura** (reversibile): il container si ferma, `instances/<slug>/` resta intatta sul Data Plane. **Eliminazione definitiva**: quella cartella viene rimossa dal Data Plane e da Chive CP, ma il backup offsite resta — è l'ultima rete di sicurezza anche dopo la cancellazione, e sopravvive pure a un problema sul Data Plane o sul Control Plane, perché vive su una terza destinazione.